Math reference

Latin American regional preparation. C++17; zero-based indices; array ranges $[l, r)$ unless stated. Choose the model, check bounds, then type. Edition 2026-10-19.

Lookup

1 Math 1
- 1.1 Recognize the mathematical reduction 1
- 1.2 Exact arithmetic 1
- 1.3 Congruences 2
- 1.4 Prime exponents and counting values 2
- 1.5 Remainder states and decimals 2
- 1.6 Approximation as a construction 3
- 1.7 Combinatorics 3
- 1.8 Probability and expectation 3
- 1.9 Proof checks 4
- 1.10 Integer geometry 4

Math (1)

1.1 Recognize the mathematical reduction

Before selecting a formula, name what the input lets you avoid. Huge decimal integers with a stated relative error may invite a bounded-prefix approximation; exact decimal text calls for integer scaling, not floating point. A rational display asks for a reduced denominator and a remainder cycle. A smallest unreachable positive sum may have a sorted-prefix invariant rather than a subset DP. A huge count of objects with a small value range often calls for frequencies and divisor classes instead of enumerating pairs. These cues are starting points: write down the exact quantity the output asks for and retain every part of the statement that changes it (ordered versus unordered, leading zeroes, strict versus inclusive error, empty choices, and modulo).

For each candidate reduction, ask four questions. What smaller object has the same answer (a prefix, a reduced denominator, a gcd class)? What invariant or identity proves that the discarded information cannot matter? What is the largest intermediate value and actual running time at the stated bound? Which helper's contract matches the inputs exactly? For instance, the decimal-shape helper handles one nonnegative fraction; it does not by itself aggregate N^2 currency pairs. Any proposed gcd-class aggregation needs its own algorithm and full-limit evidence. Likewise, factorizing a 10-digit prefix is justified by the approximation bound; that does not make arbitrary 64-bit factorization cheap.

Try the reduction on a tiny hand case and an adversarial neighbor before coding. For decimal periods, compare a terminating denominator such as 8 with one containing a coprime factor such as 14. For greedy coverage, insert a value exactly at the current boundary, then one just beyond it. If the proof needs an unstated assumption, or the tiny trace disagrees, change the model before adapting code. The worked cases in the companion chapter show this sequence: recognize, reduce, prove, trace, then map the result to a helper and its limits.

1.2 Exact arithmetic

Prelude.h

Description: Shared includes, aliases and bounds. Include before the modules.

<algorithm>, <cstdint>, <limits>, <map>, <numeric>, <optional>, <stdexcept>, <string>, <tuple>, <utility>, <vector> 17bffa, 8 lines

```
namespace finals_math {
using namespace std;
using i64 = long long;
using vi = vector<int>;
using vl = vector<i64>;
using i128 = __int128_t; // GCC/Clang

} // namespace finals_math
```

1.2.1 Floor, ceiling and bounds

Use: signed floor/ceiling division; multiples in $[L, R]$: $\max(0, \lfloor R/d \rfloor - \lceil L/d \rceil + 1)$, $d > 0$. **Pitfall:** C++ truncates toward zero. Widen *before* multiplying or subtracting: `i128(a)*b`. Helpers throw for zero denominator or overflow; 128 bits is finite.

IntegerDivision.h

Description: Checked narrowing and signed division.

"Prelude.h" cccfe3, 21 lines

```
namespace finals_math {
inline i64 narrow(i128 x) {
  if (x < numeric_limits<i64>::min() ||
      x > numeric_limits<i64>::max())
    throw overflow_error("signed 64-bit result");
  return static_cast<i64>(x);
}
// Throws for b=0 or quotient overflow.
inline i64 floor_div(i64 a, i64 b) {
  if (!b) throw invalid_argument("zero denominator");
  i128 x = a, y = b, q = x / y, r = x % y;
  if (r && ((r < 0) != (y < 0))) --q;
  return narrow(q);
}
inline i64 ceil_div(i64 a, i64 b) {
  if (!b) throw invalid_argument("zero denominator");
  i128 x = a, y = b, q = x / y, r = x % y;
  if (r && ((r < 0) == (y < 0))) ++q;
  return narrow(q);
}
} // namespace finals_math
```

1.2.2 Exact decimal input

Use: `parse_fixed(s,k)` stores $s10^k$ exactly; `digits10(x)` counts magnitude digits, including one for zero. **Requires:** $0 \le k \le 18$, integer part present, at most k fractional places, signed 64-bit result. No rounding/exponents/commas; read text, not `double`.

ExactDecimals.h

Description: Exact decimals and digit counts.

"Prelude.h", "IntegerDivision.h" 94a11e, 50 lines

```
namespace finals_math {
// Exact signed decimal; 0<=places<=18; excess digits rejected.
inline i64 parse_fixed(const string &s, int places) {
  if (places < 0 || places > 18)
    throw invalid_argument("decimal places");
  size_t j = 0;
  bool neg = false;
  if (j < s.size() && (s[j] == '+' || s[j] == '-'))
    neg = s[j++] == '-';
  i128 value = 0;
  int whole = 0, frac = 0;
  const i128 cap = i128(numeric_limits<i64>::max()) + 1;
  auto digit = [&](char c) {
    if (c < '0' || c > '9')
      throw invalid_argument("decimal digit");
    value = value * 10 + (c - '0');
    if (value > cap) throw overflow_error("decimal value");
  };
  while (j < s.size() && s[j] != '.') {
    digit(s[j++]);
    ++whole;
  }
  if (!whole) throw invalid_argument("integer part");
  if (j < s.size()) {
    ++j;
    while (j < s.size()) {
      digit(s[j++]);
      ++frac;
    }
  }
  if (frac > places)
    throw invalid_argument("too many fractional digits");
  while (frac++ < places) {
    value *= 10;
    if (value > cap) throw overflow_error("decimal value");
  }
  return narrow(neg ? -value : value);
}
// Magnitude digits: zero has one; sign excluded.
inline int digits10(i64 x) {
  i128 v = x;
  if (v < 0) v = -v;
  int d = 1;
  while (v >= 10) {
    v /= 10;
    ++d;
  }
  return d;
}
} // namespace finals_math
```

1.3 Congruences

1.3.1 Powers and inverses

Use: normalize, multiply and exponentiate modulo $m > 0$; $O(\log e)$ power, $e \geq 0$. $0^0 = 1$ before reduction. **Pitfall:** division requires $\gcd(a, m) = 1$; $a^{-1} = a^{p-2} \pmod p$ requires prime p, $p \nmid a$.

ModularArithmetic.h

Description: Modular arithmetic and binary powers.

"Prelude.h" 0b8b9c, 21 lines

```
namespace finals_math {
inline i64 norm(i64 a, i64 m) {
  if (m <= 0)
    throw invalid_argument("positive modulus required");
  a %= m;
  return a < 0 ? a + m : a;
}
inline i64 mul_mod(i64 a, i64 b, i64 m) {
  return i64(i128(norm(a, m)) * norm(b, m) % m);
}
inline i64 pow_mod(i64 a, uint64_t e, i64 m) {
  a = norm(a, m);
  i64 r = 1 % m;
  while (e) {
    if (e & 1) r = mul_mod(r, a, m);
    a = mul_mod(a, a, m);
    e >>= 1;
  }
  return r;
}
} // namespace finals_math
```

1.3.2 Bezout and shared-factor CRT

$ax + by = g = \gcd(a, b)$; $ax + by = c$ solvable iff $g \mid c$. All solutions: $(x_0 + tb/g, y_0 - ta/g)$; intersect bounds using floor/ceiling. For $a = b = 0$, handle $c = 0$ separately.

Use: merge $X = a \pmod m$, $X = b \pmod n$ even with shared factors. $g \mid (b - a)$ required; solve $(m/g)t = (b - a)/g \pmod{n/g}$, then $X = a + mt$, modulo $\operatorname{lcm}(m, n)$. **Requires:** positive moduli; nonnegative Euclid inputs. Incompatible CRT returns `nullopt`; overflowing lcm throws.

EuclidCRT.h

Description: Extended Euclid, inverses and CRT.

"Prelude.h", "IntegerDivision.h", "ModularArithmetic.h" 9f1514, 48 lines

```
namespace finals_math {
// Requires nonnegative a,b. Returns g,x,y with a*x+b*y=g.
inline tuple<i64, i128, i128> extended_gcd(i64 a, i64 b) {
  if (a < 0 || b < 0)
    throw invalid_argument("nonnegative gcd inputs");
  i128 r = a, s = b, x = 1, y = 0, u = 0, v = 1;
  while (s) {
    i128 q = r / s;
    i128 t = r - q * s;
    r = s;
    s = t;
    t = x - q * u;
    x = u;
    u = t;
    t = y - q * v;
    y = v;
    v = t;
  }
  return {i64(r), x, y};
}
inline optional<i64> inverse(i64 a, i64 m) {
  if (m <= 1)
    throw invalid_argument("inverse modulus must exceed one");
  auto [g, x, y] = extended_gcd(norm(a, m), m);
  (void)y;
  if (g != 1) return nullopt;
  return i64((x % m + m) % m);
}
struct Congruence {
  i64 residue, modulus;
};
// Merge x=a (mod m), x=b (mod n); nullopt = incompatible.
// Positive moduli. Throws if the merged lcm exceeds LLONG_MAX.
inline optional<Congruence> crt(Congruence a, Congruence b) {
  a.residue = norm(a.residue, a.modulus);
  b.residue = norm(b.residue, b.modulus);
  auto [g, x, y] = extended_gcd(a.modulus, b.modulus);
  (void)y;
  i128 delta = i128(b.residue) - a.residue;
  if (delta % g) return nullopt;
  i64 n = b.modulus / g;
  i128 t = (delta / g * x) % n;
  if (t < 0) t += n;
  i64 lcm = narrow(i128(a.modulus) * n);
  i128 r = (i128(a.residue) + i128(a.modulus) * t) % lcm;
  return Congruence{i64(r), lcm};
}
} // namespace finals_math
```

1.4 Prime exponents and counting values

1.4.1 Factorization and totient

For $n = \prod p^{e_p}$:

$$\tau(n) = \prod (e_p + 1), \quad \sigma(n) = \prod \sum_{j=0}^{e_p} p^j, \quad \phi(n) = n \prod_{p \mid n} (1 - 1/p).$$

Use: positive inputs; trial factorization $O(\sqrt{n})$, SPF preprocessing $O(V)$ time/space, factor $x \leq V$ in $O(\log x)$. **Pitfall:** repeated arbitrary 64-bit inputs need faster factorization; $n = 1$ has empty factors. Bound divisor sums/products separately.

FactorSieve.h

Description: Trial factorization, totients and sieve.

"Prelude.h" ba3fd8, 58 lines

```
namespace finals_math {
// n>0; O(sqrt(n)).
inline vector<pair<i64, int>> factor(i64 n) {
  if (n < 1) throw invalid_argument("positive factor input");
  vector<pair<i64, int>> f;
  for (i64 p = 2; p <= n / p; p += (p == 2 ? 1 : 2))
    if (n % p == 0) {
      int e = 0;
      do {
        n /= p;
        ++e;
      } while (n % p == 0);
      f.push_back({p, e});
    }
  if (n > 1) f.push_back({n, 1});
  return f;
}
inline i64 phi(i64 n) {
  i64 ans = n;
  for (auto [p, e] : factor(n)) {
    (void)e;
    ans = ans / p * (p - 1);
  }
  return ans;
}
struct Sieve {
  vi spf, primes;
  explicit Sieve(int n) {
    if (n < 0)
      throw invalid_argument("nonnegative sieve limit");
    spf.resize(size_t(n) + 1, 0);
    for (int i = 2; i <= n; ++i) {
      if (!spf[i]) {
        spf[i] = i;
        primes.push_back(i);
      }
      for (int p : primes) {
        if (p > spf[i] || p > n / i) break;
        spf[p * i] = p;
      }
    }
  }
  vector<pair<i64, int>> factors(int x) const {
    if (x < 1 || x >= int(spf.size()))
      throw invalid_argument("sieve range");
    vector<pair<i64, int>> f;
    while (x > 1) {
      int p = spf[x], e = 0;
      do {
        x /= p;
        ++e;
      } while (x > 1 && spf[x] == p);
      f.push_back({p, e});
    }
    return f;
  }
};
} // namespace finals_math
```

1.4.2 Histogram and exact gcd classes

Positive values $\leq V$: $f[x]$ = frequency, $M[d] = \sum_{k \geq 1} f[kd]$; all M cost $O(V \log V)$. For each fixed b, process divisors decreasing:

$$E_b[g] = M[g] - \sum_{\substack{h \mid b,\ g \mid h \\ h > g}} E_b[h].$$

$E_b[g]$ counts $\gcd(a, b) = g$; ordered occurrence pairs multiply by $f[b]$. **Pitfall:** zero separately; divisor subtraction adds work (e.g. $\sum_b \tau(b)^2$).

1.5 Remainder states and decimals

1.5.1 Digits as a small state

Append d: $r' = (10r + d) \bmod m$. **Use:** DP (position, r), $O(10Lm)$, only when equal states have identical legal suffixes. Add other distinguishing history to state. **Pitfall:** leading zeros; widen $10r$. Smallest reconstruction tries digits ascending with suffix feasibility, not counts alone.

1.5.2 Period and rational display

$\mathrm{ord}_m(a) \mid \phi(m)$ for $m \geq 2$, $\gcd(a, m) = 1$. **Use:** start $k = \phi(m)$; divide prime factors p while $a^{k/p} = 1 \pmod m$.
Trial-factorization cost applies.

Reduce $q = b/\gcd(a, b) = 2^u 5^v r$, $\gcd(r, 10) = 1$:

$$D(a, b) = \mathrm{digits}(\lfloor a/b \rfloor) + \max(u, v) + P(r),$$

$P(1) = 0$, otherwise $P(r) = \mathrm{ord}_r(10)$. **Why:** long division removes the $2, 5$ prefix; recurring remainders then require $10^k = 1 \pmod r$. **Requires:** $a \geq 0, b > 0$; canonical decimal, no point/overbar/trailing zeros. 2025 D aggregation: gcd classes for fractions; suffix counts at $a \geq 10b, 100b, \ldots$ for integer digits. Maximum-size aggregation unbenchmarked.

DecimalPeriod.h

Description: Multiplicative order and decimal shape.

"Prelude.h", "ExactDecimals.h", "ModularArithmetic.h", "FactorSieve.h" 4d9be3, 35 lines

```
namespace finals_math {
inline i64 order_mod(
    i64 a,
    i64 m) { // m>=2, gcd(a,m)=1; trial division cost applies.
  if (m < 2 || gcd(norm(a, m), m) != 1)
    throw invalid_argument("order requires a unit");
  i64 k = phi(m);
  for (auto [p, e] : factor(k)) {
    (void)e;
    while (k % p == 0 && pow_mod(a, k / p, m) == 1) k /= p;
  }
  return k;
}
struct DecimalShape {
  int integer_digits, nonrepeating;
  i64 period;
};
// a>=0, b>=1; period computation may be costly.
inline DecimalShape decimal_shape(i64 a, i64 b) {
  if (a < 0 || b < 1)
    throw invalid_argument("nonnegative fraction");
  i64 q = b / gcd(a, b);
  int u = 0, v = 0;
  while (q % 2 == 0) {
    q /= 2;
    ++u;
  }
  while (q % 5 == 0) {
    q /= 5;
    ++v;
  }
  return {digits10(a / b), max(u, v),
          q == 1 ? 0 : order_mod(10, q)};
}
} // namespace finals_math
```

1.6 Approximation as a construction

1.6.1 Fuzzy Factorization (2025 F)

Keep ten leading digits: $X = m10^e + r$, $0 \leq r < 10^e$, $m \geq 10^9$. Choose $Y = m10^e$: $|X - Y|/X < 1/m \leq 10^{-9}$. **Use:** factor $m < 10^{10}$; add e to exponents of $2, 5$. At most ten digits: exact factorization. $O(|X| + \sqrt{m})$. **Requires:** positive canonical decimal text. Empty map means one; merge primes and adapt output format. Different tolerance needs a new bound.

FuzzyFactorization.h

Description: Fuzzy factorization construction.

"Prelude.h", "FactorSieve.h" 41a1d5, 21 lines

```
namespace finals_math {
// Positive decimal, no leading zero; empty factors means Y=1.
inline map<i64, int> fuzzy_factors(const string &s) {
  if (s.empty() || s[0] == '0')
    throw invalid_argument("positive canonical decimal");
  for (char c : s)
    if (c < '0' || c > '9')
      throw invalid_argument("decimal digits");
  i64 m = 0;
  int keep = int(min<size_t>(10, s.size()));
  for (int i = 0; i < keep; ++i) m = m * 10 + s[i] - '0';
  int e = int(s.size()) - keep;
  map<i64, int> f;
  for (auto [p, k] : factor(m)) f[p] = k;
  if (e) {
    f[2] += e;
    f[5] += e;
  }
  return f;
}
} // namespace finals_math
```

1.7 Combinatorics

1.7.1 Sample objects and binomial coefficients

Disjoint cases add; exactly A choices then B each multiply. Ordered distinct: $n!/(n-k)!$; subsets: $\binom{n}{k}$; ordered repetition: n^k. Stars/bars, $k \geq 1$: nonempty bins $\binom{n-1}{k-1}$ $(n \geq k)$; empty allowed $\binom{n+k-1}{k-1}$. Pascal: $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ (any modulus). **Use:** factorial tables, $O(N + \log p)$ build, $O(1)$ query. **Requires:** prime p, $N < p$; primality unchecked. No Lucas/composite support.

Combinations.h

Description: Combinations under a prime modulus.

"Prelude.h", "ModularArithmetic.h" 10ccf0, 25 lines

```
namespace finals_math {
// Requires prime p and n<p; primality is unchecked.
struct PrimeCombinations {
  i64 p;
  vl fac, invfac;
  PrimeCombinations(int n, i64 prime) : p(prime) {
    if (n < 0 || p <= 1 || n >= p)
      throw invalid_argument("need 0<=n<prime");
    fac.resize(n + 1, 1);
    invfac.resize(n + 1, 1);
    for (int i = 1; i <= n; ++i)
      fac[i] = mul_mod(fac[i - 1], i, p);
    invfac[n] = pow_mod(fac[n], p - 2, p);
    for (int i = n; i > 0; --i)
      invfac[i - 1] = mul_mod(invfac[i], i, p);
  }
  i64 choose(int n, int k) const {
    if (n < 0 || n >= int(fac.size()))
      throw invalid_argument("choose range");
    if (k < 0 || k > n) return 0;
    return mul_mod(fac[n],
                   mul_mod(invfac[k], invfac[n - k], p), p);
  }
};
} // namespace finals_math
```

1.7.2 Contributions and complements

$$\sum_{\text{subarrays}} \sum a_i = \sum_i a_i(i+1)(n-i),$$

$$\sum_{i<j}(a_i + a_j) = (n-1)\sum a_i, \quad \sum_{i<j}(a_i - a_j)^2 = n\sum a_i^2 - (\sum a_i)^2.$$

At least one good = total − zero good. Bad run length L: $L(L+1)/2$ all-bad substrings. **Pitfall:** wide products; circular boundaries/all-bad case separately.

1.7.3 Inversions and repeated copies

Use: strict inversions $i < j, a_i > a_j$; compression/Fenwick, $O(n \log n)$ time, $O(n)$ space. Equal values do not count.

$$\mathrm{inv}(S^K) = K \text{ within} + \binom{K}{2} \text{ cross}.$$

cross $= \#\{(u, v) : S_u > S_v\}$ with no within-copy index restriction (2023 I). Widen/modularize products.

Inversions.h

Description: Strict inversions by compressed Fenwick tree.

"Prelude.h", "IntegerDivision.h" 5e519b, 21 lines

```
namespace finals_math {
// Counts i<j with a[i]>a[j]; O(n log n); input unchanged.
inline i64 inversions(vl a) {
  auto vals = a;
  sort(vals.begin(), vals.end());
  vals.erase(unique(vals.begin(), vals.end()), vals.end());
  vl bit(vals.size() + 1);
  i64 answer = 0, seen = 0;
  for (i64 x : a) {
    int k = int(lower_bound(vals.begin(), vals.end(), x) -
                vals.begin()) +
            1;
    i64 le = 0;
    for (int j = k; j; j -= j & -j) le += bit[j];
    answer = narrow(i128(answer) + seen - le);
    ++seen;
    for (int j = k; j < int(bit.size()); j += j & -j) ++bit[j];
  }
  return answer;
}
} // namespace finals_math
```

1.8 Probability and expectation

$$\mathbb{E}[X] = \sum_x x \Pr(X = x), \quad \mathbb{E}[\sum I_j] = \sum \Pr(I_j = 1).$$

Linearity needs no independence; $\Pr(A \cap B) = \Pr(A)\Pr(B \mid A)$. Ordered occurrence pairs: N^2 with replacement, $N(N-1)$ without. $E = c + qE + \sum p_j E_j \Rightarrow E = (c + \sum p_j E_j)/(1-q)$, $q < 1$. **Pitfall:** modular A/B requires invertible reduced denominator; never convert rounded decimals.
$\mathrm{Var}(X) = E[X^2] - E[X]^2$.

1.9 Proof checks

Invariant: initialize, preserve, then prove sufficiency by construction. Termination: decreasing nonnegative integer. Binary search: monotone predicate. Greedy: feasibility-preserving exchange. Tiny brute force finds counterexamples. $\sum i = n(n+1)/2$; $\sum_{i=0}^{k} r^i = (r^{k+1}-1)/(r-1)$, $r \neq 1$. Modular division needs cancellation/invertibility; error uses original denominator.

1.9.1 Greedy coverage: smallest missing subset sum

Positive integers sorted ascending; initially $R = 0$ and every sum in $[0, R]$ is reachable. If $x > R + 1$, answer $R + 1$; otherwise extend $R \leftarrow R + x$. After all values answer $R + 1$. Proof: old sums $[0, R]$ and sums using x, $[x, x + R]$, have no integer gap iff $x \leq R + 1$. Masters Mexico 2022 H; $O(n \log n)$; total plus one must fit.

```
long long missing(std::vector<long long> a) {
  std::sort(a.begin(), a.end());
  long long next = 1;
  for (auto x : a) {
    assert(x > 0);
    if (x > next) break;
    next += x;
  }
  return next;
}
```

1.10 Integer geometry

$u \cdot v = u_x v_x + u_y v_y$; $u \times v = u_x v_y - u_y v_x$.
orient$(A, B, C) = (B - A) \times (C - A)$: positive left, zero collinear. $2A_{\triangle} = |\,\text{orient}\,|$; compare squared distances. **Use:** exact predicates; segment membership = collinear + inclusive bounds. Simple polygon in boundary order: $2A = |\sum P_i \times P_{i+1}|$.
Requires: helper coordinates $|x|, |y| \leq 10^{18}$; polygon sum needs its own bound. Compare orientation signs, not multiplied crosses; no universal floating EPS.

IntegerGeometry.h

Description: Exact integer geometry.

"Prelude.h" 45bdbc, 17 lines

```
namespace finals_math {
struct Point {
  i64 x, y;
};
// |coordinate|<=1e18 keeps these predicates within i128.
inline i128 orient(Point a, Point b, Point c) {
  return (i128(b.x) - a.x) * (i128(c.y) - a.y) -
         (i128(b.y) - a.y) * (i128(c.x) - a.x);
}
inline i128 dot(Point a, Point b) {
  return i128(a.x) * b.x + i128(a.y) * b.y;
}
inline i128 dist2(Point a, Point b) {
  i128 x = i128(a.x) - b.x, y = i128(a.y) - b.y;
  return x * x + y * y;
}
} // namespace finals_math
```